

M3 – Critical Evaluation and Transfer: Low Power Scheduling for High-Level Synthesis

Rionela Kovaci
HW/SW Codesign Seminar
Hochschule Hamm-Lippstadt (HSHL)
Hamm, Germany

MILESTONE 3: CRITICAL EVALUATION AND TRANSFER

1. Concrete strengths

- The partitioning idea is **architecture-independent**: the method works on any CDFG and is not tied to a specific hardware platform.
- The **cost function** (TotalPowerDelay) is well defined and accounts for guard overhead, so the method does not blindly add partitions that cost more than they save.
- Integration into existing HLS flow is **non-invasive**: the Power Scheduler receives a pre-scheduled CDFG and returns a partitioned one; existing schedulers do not need to change.
- The approach explicitly targets **self-controlled architectures** with distributed control, which are common in modern embedded SoC designs.
- Results are **validated with Synopsys Power Compiler** on synthesisable VHDL, giving industrial-grade credibility to the 15% saving.

2. Concrete limitations and hidden assumptions

- **Clique search is NP-hard**. For large CDFGs with many paths, the search may become impractical. No approximation strategy is described.
- **Only one benchmark** (MPEG-2 chrominance conversion) is evaluated. This is a data-path-dominated example; it is unclear how the method performs on control-dominated or mixed designs.
- **Guard overhead is always present**. The control unit of each partition is always active. If partitions are too small, the overhead exceeds the saving.
- **Static power is only partly addressed**. Resource sharing reduces leakage by lowering transistor count, but no voltage scaling or power gating is applied.
- **Scheduling is assumed to be done**. The Power Scheduler cannot improve a poor initial schedule; sub-optimal scheduling upstream limits its effectiveness.
- **No timing analysis of guard latency**. Scheduling control nodes one step earlier adds latency; the impact on timing-constrained designs is not discussed.

3. Scalability

- The compatibility graph has $\mathcal{O}(p^2)$ edges where p is the number of paths. This is acceptable for small to medium designs but may be problematic for complex SoCs with hundreds of concurrent operations.
- Clique search complexity grows exponentially in the worst case. Real designs may require approximation algorithms or time limits.
- Memory overhead of storing full path traces and the compatibility graph is not discussed.

4. Where the approach is suitable

- **Automotive control units**: control-dominated, many mutually exclusive execution branches, strict power budgets, and self-controlled architectures are common.
- **Wireless sensor nodes**: small chips, low power mandatory, control-flow intensive protocol handling.
- **Signal processing pipelines** with alternative paths (e.g. MPEG, H.264 codecs) where large portions of the datapath are conditionally active.

5. Where the approach is unsuitable

- **Hard real-time systems** with strict latency bounds: the extra clock step from early control node scheduling may violate deadlines.
- **Purely data-dominated designs** (e.g. dense matrix multiplication): all paths are always active, so there are no mutually exclusive paths to partition and no power saving opportunity.
- **Very large CDFGs** where clique search becomes infeasible without approximation.

6. Possible extensions and open questions

- **Combine with DVFS**: each partition could run at a different voltage/frequency when active, multiplying the power saving.
- **Approximate clique search**: replacing exact clique search with a greedy or genetic algorithm would improve scalability to large designs.
- **Static power extension**: adding power gating inside partitions would also reduce leakage, addressing the current limitation.

- **Multi-benchmark validation:** testing on automotive, network protocol, and real-time control benchmarks would establish generalisability.
- **Open question:** can the partitioning approach be applied online (at runtime) in reconfigurable hardware, adapting partitions to current workload?

REFERENCES

- [1] A. Rettberg, C. Romaus, and H. Franke, "A new design partitioning approach for low power high-level synthesis," in *Proc. IEEE*, 2006. [Online]. Available: <https://ieeexplore.ieee.org/document/1581203>
- [2] E. Musoll and J. Cortadella, "Scheduling and resource binding for low power," in *Proc. 8th Int. Symp. System Synthesis*, 1995, pp. 104–109.
- [3] W.-T. Shiue and C. Chakrabarti, "ILP-based scheme for low power scheduling and resource binding," in *Proc. IEEE Int. Symp. Circuits and Systems (ISCAS)*, Geneva, Switzerland, May 2000, pp. III-279–III-282.
- [4] K. S. Khouri, G. Lakshminarayana, and N. K. Jha, "IMPACT: A high-level synthesis system for low power control-flow intensive circuits," in *Proc. Design Automation Conf.*, 1998.